



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ»

Домашнее задание

ДИСЦИПЛИНА: «Программирование на основе классов и шаблонов»

Выполнил: студент гр. ИУ5-24Б

(Подпись) (Лимаренко Д. А.)
(Ф.И. О.)

Проверил:

(Подпись) (Волосова А. В.)
(Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Постановка задачи

Задание. Выполнить обработку изображения, выбранного из списка файлов, по следующей схеме:

1. Выворачивать изображение относительно горизонтальных и вертикальных линий. Если нужно, то развернуть изображение в горизонтальное положение.
2. Определить по изображению размерность и длительность исследования.
3. Выделить первый график.
4. Выделить второй график.
5. Построить временные ряды первого и второго графика.
6. Сохранить результаты в виде csv файла.

Задание заключается в обработке диаграммы медицинского исследования для последующей аналитики.

По результатам обработки создается .csv - файл, который затем может использоваться в экспертных медицинских системах и других приложениях, как вспомогательная информация для врачей (постановка, подтверждение диагноза и т.п.), а также для различной аналитики.

Выполнение работы

Был создан репозиторий на GitHub:
https://github.com/DariaLimarenko/ctg_project

На рисунке 1 представлен репозиторий на GitHub:

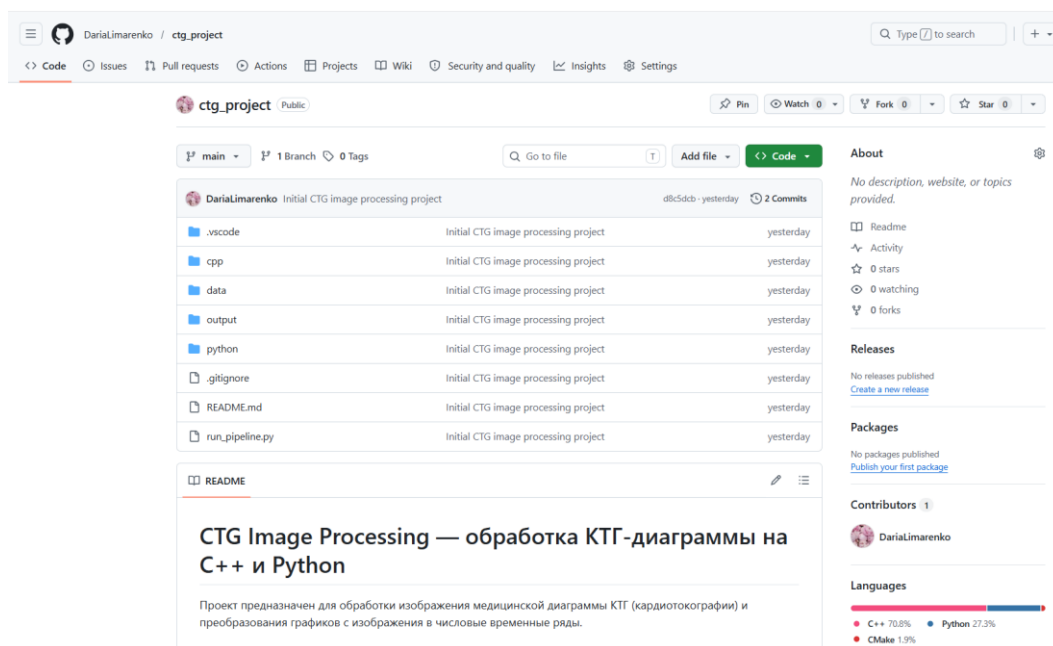


Рисунок 1 – Репозиторий на GitHub

Для выполнения задания было выбрано изображение 14-3, представленное на рисунке 2.

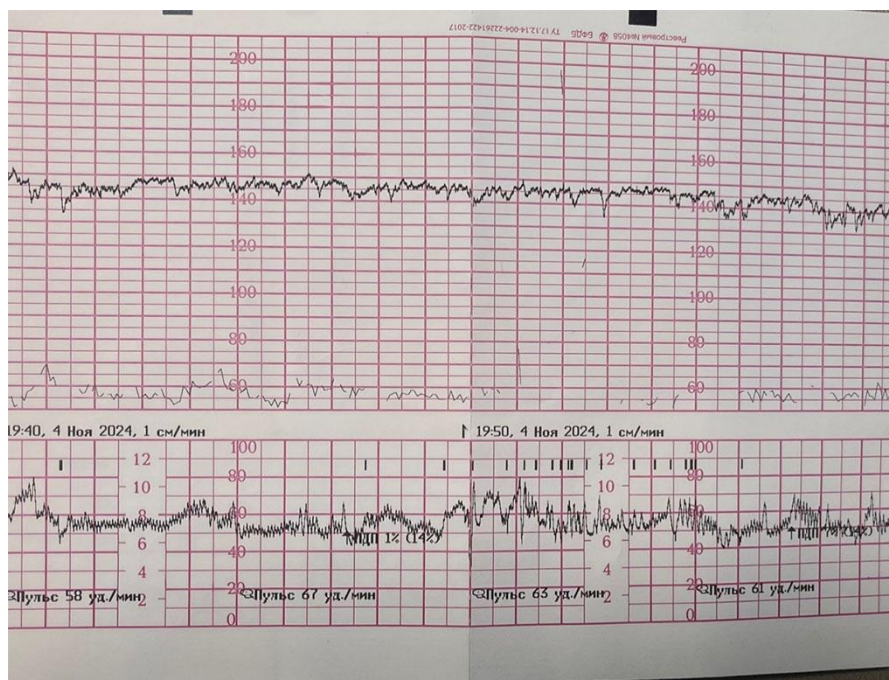


Рисунок 2 – Изображение 14-3

Был разработан проект для обработки изображения медицинского исследования. В качестве исходных данных используется изображение КТГ-диаграммы, на которой представлены два графика: верхний график частоты сердечных сокращений и нижний график токограммы. Основная задача проекта состоит в том, чтобы автоматически обработать изображение, выделить оба графика, восстановить по ним временные ряды и сохранить полученные данные в CSV-файл.

Проект был реализован на двух языках программирования: C++ и Python. C++ используется для основной обработки изображения, так как он хорошо подходит для задач компьютерного зрения и эффективной работы с изображениями. Python используется для визуализации результатов, чтения CSV-файла, сглаживания временных рядов и построения итогового графика.

Для обработки изображения в C++ применяется библиотека OpenCV. Она используется для загрузки изображения, преобразования его в оттенки серого, поиска линий, выравнивания изображения, выделения областей графиков и построения бинарных масок. Для Python-части используются библиотеки pandas, numpy, matplotlib и scipy. Они позволяют удобно работать с табличными данными, выполнять дополнительную обработку сигналов и строить итоговые графики.

Также в проект был добавлен этап постобработки результатов. Он выполняется Python-скриптом `postprocess_results.py`. Данный скрипт создаёт отдельные CSV-файлы для первого и второго графика, формирует итоговый отчёт `final_report.md` и строит изображение `combined_overlay.png`, на котором найденные элементы графиков накладываются на исходную диаграмму.

Структура проекта организована таким образом, чтобы разделить исходные данные, программный код и результаты работы. В папке `data` находится исходное изображение `my_ctg.jpg`. В папке `cpp` размещены исходные файлы C++-программы. В папке `python` находятся Python-скрипты для визуализации, анализа CSV и дополнительной постобработки результатов. В папке `output` сохраняются результаты обработки: изображение после коррекции наклона, маски графиков, итоговый CSV-файл, отдельные CSV-файлы для каждого графика, построенный итоговый график, автоматически сформированный отчёт и изображение с наложением найденных графиков на исходную диаграмму. Структура проекта представлена на рисунках 3 и 4.

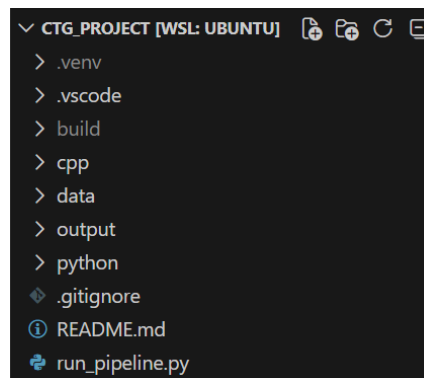


Рисунок 3 – Структура проекта

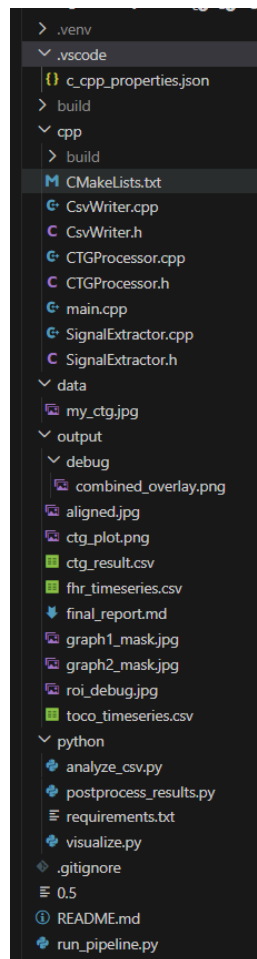


Рисунок 4 – Структура проекта

В C++-части проекта основная логика реализована в классе CTGProcessor. Этот класс отвечает за загрузку изображения, выравнивание, определение размерности, выделение областей графиков и формирование временных рядов. Отдельно реализован модуль SignalExtractor, который отвечает за извлечение сигнала из области изображения. Также присутствует модуль CsvWriter, который сохраняет полученные данные в CSV-файл.

Работа программы начинается с загрузки исходного изображения. Программа получает путь к файлу data/my_ctg.jpg и пытается открыть его с помощью OpenCV. Если файл отсутствует, повреждён или имеет нулевой размер, программа сообщает об ошибке. В ходе выполнения проекта изображение было успешно загружено, его размер составил 1024 x 774 пикселя.

После загрузки программа проверяет ориентацию изображения. Если изображение расположено вертикально, оно разворачивается в горизонтальное положение. Это необходимо, потому что медицинская диаграмма должна обрабатываться как горизонтальная временная запись: время идёт слева направо, а

значения графиков откладываются по вертикали. В данном случае изображение уже было в горизонтальной ориентации, поэтому дополнительный поворот на 90 градусов не потребовался.

Следующим этапом выполняется выравнивание изображения относительно линий сетки. Поскольку исходная диаграмма была получена с фотографии, она может иметь небольшой наклон. Для корректного извлечения графиков необходимо, чтобы горизонтальные и вертикальные линии сетки были выровнены. Программа переводит изображение в оттенки серого, выделяет границы и с помощью преобразования находит линии сетки. По найденным линиям вычисляется угол наклона изображения.

В результате обработки был определён угол наклона около 0.999997 градуса. После этого изображение было повернуто на найденный угол, чтобы привести его к более правильному горизонтальному положению. Результат данного этапа сохраняется в файл `output/aligned.jpg`. Этот файл нужен для визуальной проверки коррекции наклона изображения перед дальнейшей обработкой. Следует отметить, что в текущей версии программы выполняется именно коррекция небольшого угла поворота по линиям сетки, а не полное перспективное выравнивание листа, как при сканировании документа.

После выравнивания программа определяет размерность изображения и длительность исследования. Для этого анализируется сетка диаграммы. На исходном изображении указана скорость бумаги 1 см/мин, поэтому по горизонтальному размеру сетки можно оценить длительность записи. Программа определяет шаг малой клетки в пикселях и рассчитывает длительность исследования. В результате был найден шаг малой клетки около 7 px, а длительность исследуемого фрагмента составила примерно 14.6286 минуты.

Далее выполняется разделение изображения на две области интереса. Верхняя часть изображения соответствует первому графику, то есть FHR — частоте сердечных сокращений плода. Нижняя часть соответствует второму графику, то есть Тосо — токограмме или маточной активности. Между ними находится область с временными отметками и служебными подписями. Программа определяет строку разделения между графиками, в данном случае она была определена как 467 px.

После определения строки разделения изображение делится на две области. Для верхнего графика была выделена область размером 1004 x 417 px, а для нижнего графика — область размером 1004 x 257 px. Для проверки корректности разделения программа сохраняет изображение `output/roi_debug.jpg`, на котором показано, как исходная диаграмма была разделена на две части.

Первый график, FHR, обрабатывается отдельно. Программа берёт верхнюю область изображения и ищет в ней тёмные пиксели, соответствующие линии графика. Сетка и фон отбрасываются, а найденные элементы линии сохраняются в виде бинарной маски. В этой маске чёрный цвет соответствует фону, а белый цвет соответствует пикселям, которые программа распознала как часть графика. Результат сохраняется в файл `output/graph1_mask.jpg`.

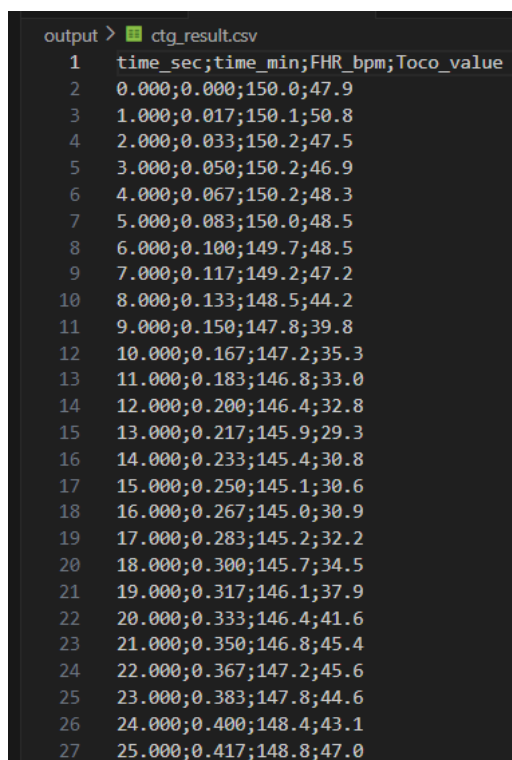
По маске первого графика программа извлекает числовой сигнал. Для каждого столбца изображения определяется положение линии графика по вертикали. Затем координата пикселя переводится в значение частоты сердечных сокращений плода. В результате было извлечено 1004 точки сигнала FHR. Эти точки затем используются для построения временного ряда.

Второй график, Тосо, обрабатывается аналогичным образом. Программа берёт нижнюю область изображения, выделяет тёмные пиксели и формирует бинарную маску. Результат сохраняется в файл `output/graph2_mask.jpg`. На этой маске, кроме линии графика, могут присутствовать подписи, цифры и служебные отметки, потому что нижняя часть исходной диаграммы содержит больше текстовой информации. Это является особенностью и ограничением текущего алгоритма.

Несмотря на наличие лишних элементов в маске второго графика, программа также извлекает временной ряд Тосо. Для каждого столбца определяется координата линии, после чего она переводится в числовое значение токограммы. В результате для второго графика также было извлечено 1004 точки сигнала.

После выделения обоих графиков программа строит временные ряды. Для этого создаётся временная ось, которая содержит время в секундах и минутах. Значения первого графика сохраняются в колонку `FHR_bpm`, а значения второго графика — в колонку `Тосо_value`. Таким образом, для каждого момента времени формируется строка с двумя значениями сигналов.

Основным результатом C++ части является CSV-файл `output/ctg_result.csv`. Он содержит следующие колонки: `time_sec`, `time_min`, `FHR_bpm` и `Toco_value`. Колонка `time_sec` хранит время в секундах, `time_min` — время в минутах, `FHR_bpm` — частоту сердечных сокращений плода в ударах в минуту, а `Toco_value` — значение токограммы. На рисунке 5 представлен пример CSV-файла.



```
output > ctg_result.csv
1 time_sec;time_min;FHR_bpm;Toco_value
2 0.000;0.000;150.0;47.9
3 1.000;0.017;150.1;50.8
4 2.000;0.033;150.2;47.5
5 3.000;0.050;150.2;46.9
6 4.000;0.067;150.2;48.3
7 5.000;0.083;150.0;48.5
8 6.000;0.100;149.7;48.5
9 7.000;0.117;149.2;47.2
10 8.000;0.133;148.5;44.2
11 9.000;0.150;147.8;39.8
12 10.000;0.167;147.2;35.3
13 11.000;0.183;146.8;33.0
14 12.000;0.200;146.4;32.8
15 13.000;0.217;145.9;29.3
16 14.000;0.233;145.4;30.8
17 15.000;0.250;145.1;30.6
18 16.000;0.267;145.0;30.9
19 17.000;0.283;145.2;32.2
20 18.000;0.300;145.7;34.5
21 19.000;0.317;146.1;37.9
22 20.000;0.333;146.4;41.6
23 21.000;0.350;146.8;45.4
24 22.000;0.367;147.2;45.6
25 23.000;0.383;147.8;44.6
26 24.000;0.400;148.4;43.1
27 25.000;0.417;148.8;47.0
```

Рисунок 5 – Пример CSV-файла `ctg_result.csv`

После создания CSV-файла запускается Python-часть проекта. Python-скрипт `visualize.py` читает файл `output/ctg_result.csv`, проверяет наличие нужных колонок и выполняет дополнительное сглаживание данных. Сглаживание необходимо для того, чтобы итоговые графики были более читаемыми и менее шумными.

После чтения и обработки CSV Python строит два графика. Первый график отображает частоту сердечных сокращений плода во времени. На нём также показаны ориентировочные линии нижней и верхней границы нормы. Вторым графиком отображаются значения токограммы во времени. Итоговая визуализация сохраняется в файл `output/ctg_plot.png`.

Итоговый график `ctg_plot.png` строится по данным из CSV-файла. Перед построением выполняется дополнительное сглаживание временных рядов, чтобы уменьшить влияние шума, возникающего при выделении линии графика с

изображения. Поэтому итоговая линия выглядит более плавной, чем исходная кривая на фотографии. При этом основная динамика сигналов сохраняется.

В ходе выполнения проекта были получены следующие основные результаты: `aligned.jpg`, `roi_debug.jpg`, `graph1_mask.jpg`, `graph2_mask.jpg`, `ctg_result.csv`, `fhr_timeseries.csv`, `toco_timeseries.csv`, `ctg_plot.png`, `final_report.md` и `combined_overlay.png`. Файл `aligned.jpg` содержит изображение после коррекции небольшого угла наклона по линиям сетки. Файл `roi_debug.jpg` показывает разделение изображения на два графика. Файлы `graph1_mask.jpg` и `graph2_mask.jpg` показывают процесс выделения линий графиков. Файл `ctg_result.csv` содержит итоговые числовые данные обоих временных рядов. Файлы `fhr_timeseries.csv` и `toco_timeseries.csv` содержат временные ряды первого и второго графика отдельно. Файл `ctg_plot.png` содержит визуализацию восстановленных временных рядов. Файл `final_report.md` содержит краткий автоматически сформированный отчёт по результатам обработки. Файл `combined_overlay.png` показывает наложение найденных элементов графиков на исходное изображение.

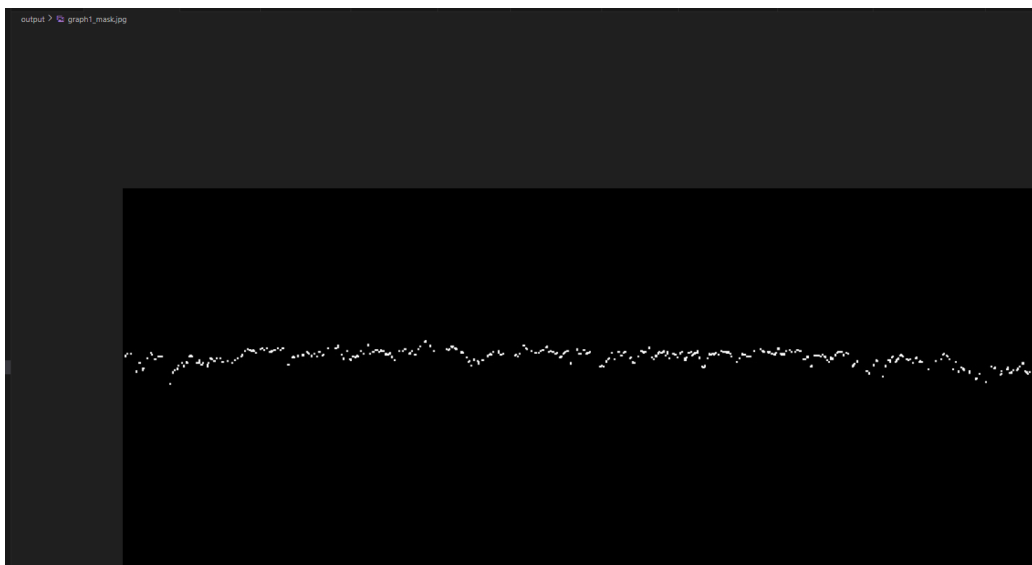


Рисунок 6 – Изображение `graph1_mask.jpg`

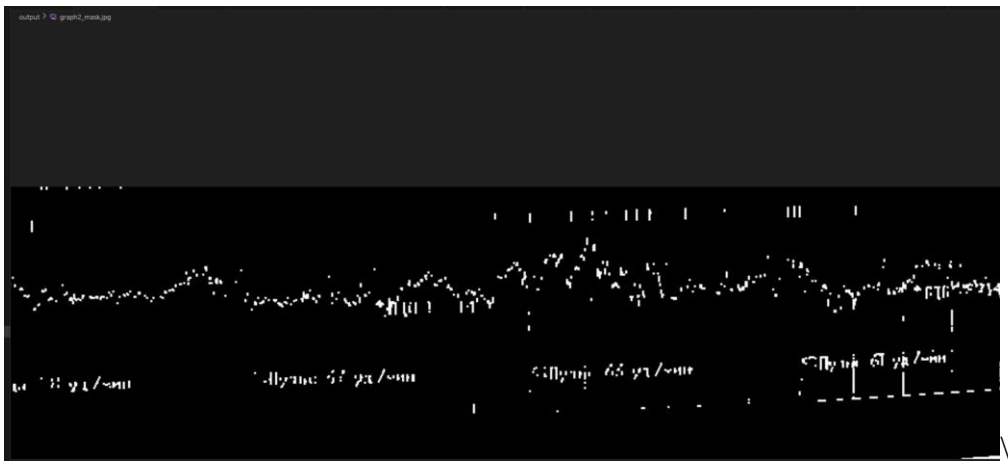


Рисунок 7 – Изображение graph2_mask.jpg



Рисунок 8 – Изображение roi_debug.jpg

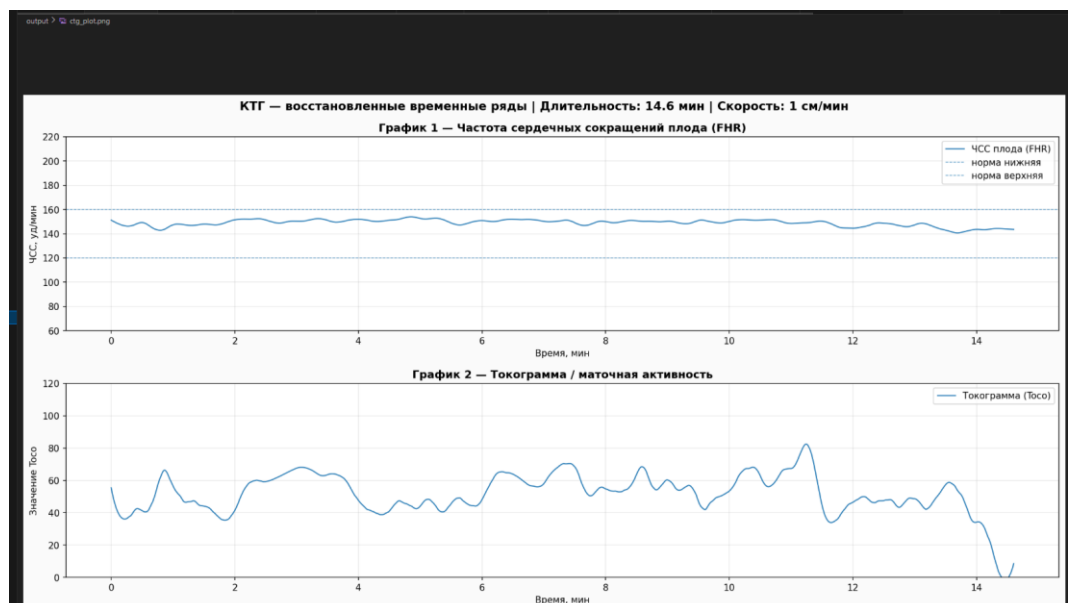


Рисунок 9 – Изображение ctg_plot.png

После выполнения основной обработки изображения и построения итогового графика был реализован дополнительный этап постобработки. Он выполняется с помощью Python-скрипта `postprocess_results.py`. Данный этап был добавлен для того, чтобы сделать результаты более удобными для анализа и проверки.

На этапе постобработки основной CSV-файл `output/ctg_result.csv` разделяется на два отдельных файла. Файл `output/fhr_timeseries.csv` содержит только временной ряд первого графика FHR, а файл `output/toco_timeseries.csv` содержит только временной ряд второго графика Тосо. Такое разделение удобно, если необходимо анализировать каждый график отдельно или передавать отдельные временные ряды в другие программы.

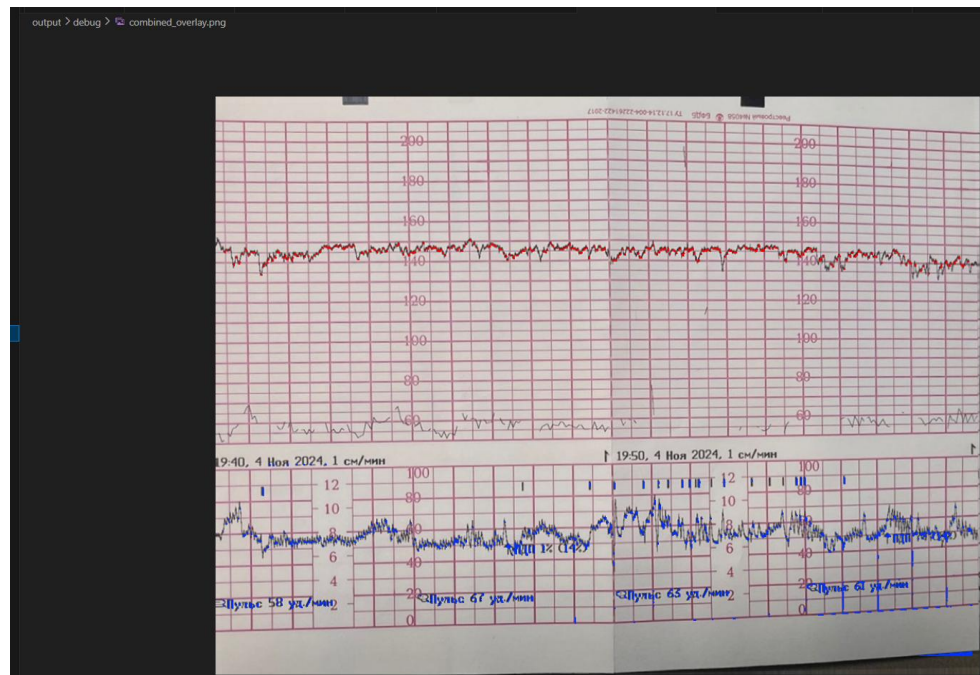
Также автоматически создаётся файл `output/final_report.md`. В нём сохраняется краткая информация о результате обработки: длительность исследования, количество строк в CSV-файле, минимальные, максимальные и средние значения сигналов FHR и Тосо, а также список созданных выходных файлов. Этот файл позволяет быстро проверить результат работы программы без просмотра исходного кода.

```
output > final_report.md > ** # Итоговый отчёт обработки КТГ > ** ## Основные выходные файлы
1
2 # Итоговый отчёт обработки КТГ
3
4 ## Общая информация
5
6 Выполнена автоматизированная обработка изображения медицинской диаграммы КТГ.
7
8 Программа выполнила загрузку изображения, выравнивание по сетке, выделение двух графиков, построение временных рядов и сохранение результата в CSV-файл.
9
10 ## Разрешение изображений
11
12 - Ширина изображения: '1024 px'
13 - Высота изображения: '774 px'
14
15 ## Длительность исследования
16
17 - Длительность в секундах: '876.0 сек'
18 - Длительность в минутах: '14.60 мин'
19 - Количество строк в CSV: '877'
20
21 ## Первый график – FHR
22
23 Первый график соответствует частоте сердечных сокращений плода.
24
25 - Минимальное значение FHR: '140.4 уд/мин'
26 - Максимальное значение FHR: '154.9 уд/мин'
27 - Среднее значение FHR: '149.0 уд/мин'
28 - Маска первого графика: 'output/graph1_mask.jpg'
29 - Отдельный CSV первого графика: 'output/fhr_timeseries.csv'
30
31 ## Второй график – Тосо
32
33 Второй график соответствует токограмме / маточной активности.
34
35 - Минимальное значение Тосо: '1.2'
36 - Максимальное значение Тосо: '86.0'
37 - Среднее значение Тосо: '51.6'
38 - Маска второго графика: 'output/graph2_mask.jpg'
39 - Отдельный CSV второго графика: 'output/toco_timeseries.csv'
40
```

Рисунок 10 – Итоговый отчёт `final_report.md`

Кроме того, создаётся файл `output/debug/combined_overlay.png`. Это изображение используется для визуальной проверки качества выделения графиков. На нём найденные элементы первого графика FHR отображаются красным цветом, а найденные элементы второго графика Тосо — синим цветом. По данному

изображению можно оценить, насколько хорошо программа сопоставила найденные линии с исходной диаграммой.



*Рисунок 11 – Наложение найденных графиков на исходное изображение
combined_overlay.png*

На overlay-изображении видно, что верхний график FHR выделяется достаточно точно: красная линия проходит вдоль исходной кривой. Для нижнего графика Тосо часть текстовых подписей и служебных отметок также может попадать в маску, поскольку нижняя область диаграммы содержит много тёмных элементов. Это является ограничением текущего алгоритма, однако основной ход второго графика сохраняется и используется для построения временного ряда.

Вывод: в результате выполнения работы была создана программа, которая анализирует изображение медицинской КТГ-диаграммы. Она загружает изображение, выполняет коррекцию наклона относительно линий сетки, определяет размерность и длительность исследования, выделяет первый и второй графики, строит по ним временные ряды и сохраняет результат в CSV-файл.

Дополнительно программа создаёт отладочные изображения и итоговую визуализацию, что позволяет проверить корректность выполнения каждого этапа обработки. В ходе доработки проекта был добавлен этап постобработки: формируются отдельные CSV-файлы `fhr_timeseries.csv` и `toco_timeseries.csv`, автоматически создаётся отчёт `final_report.md`, а также строится изображение `combined_overlay.png` с наложением найденных графиков на исходную диаграмму.

Таким образом, проект демонстрирует практическое применение методов компьютерного зрения для обработки медицинских диаграмм. Использование C++ и Python позволило разделить задачи: C++ отвечает за эффективную обработку изображения и извлечение данных, а Python — за анализ, визуализацию, сглаживание и постобработку результата. Итоговый CSV-файл может быть использован для дальнейшей обработки, аналитики и демонстрации работы алгоритма.

Разработанная программа является учебным прототипом и не предназначена для самостоятельной медицинской диагностики. Полученные данные могут рассматриваться только как вспомогательная информация для последующего анализа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электроннобиблиотечная система. — URL: <https://e.lanbook.com/book/103505>
 2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электроннобиблиотечная система. — URL: <https://e.lanbook.com/book/5161>
 3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-570384577-6.
 4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002.
<http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html>
 5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.
 6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. <http://progbook.ru/c/737-podbelskii-programmiovaniye-na-yazyke-si.html>.
 7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006.
<http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedevnvinformatika-bazovyy-kurs.html>
 8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. <https://num-meth.ru/index.php/journal/index>
- Дополнительные материалы**

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. <http://razym.ru/naukaobraz/obrazov/181547aho-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html>
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на С++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. <http://rutracker.org/forum/viewtopic.php?t=533181>
4. Джосьютис Н. С++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в С++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 594723302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. С++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. С/С++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): <https://education.vk.company/students>